NUMPY PROGRAMS

1.Write a Python program to:

- Import NumPy.
- Create a NumPy array containing 10 integers.
- Display the array.
- Display its data type.

```
import numpy as np


# Create a NumPy array containing 10 integers

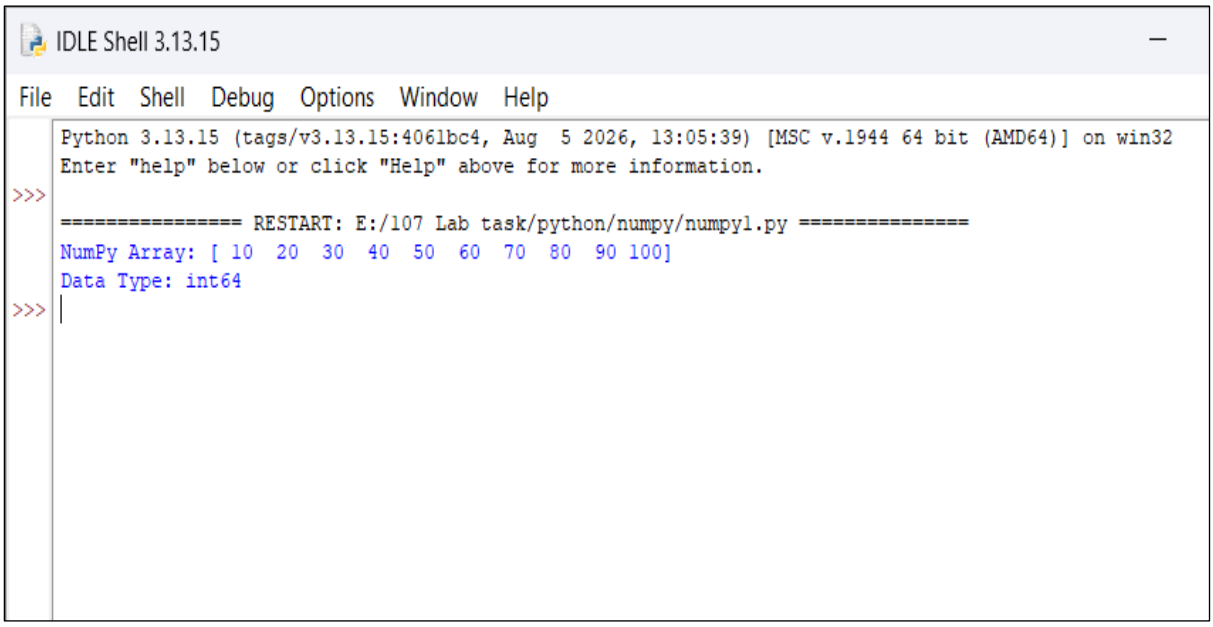
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])


# Display the array

print("NumPy Array:", arr)


# Display the data type

print("Data Type:", arr.dtype)
```



2. Create a NumPy array containing 10 numbers and perform:

- Addition of 5 to every element
- Subtraction of 2 from every element
- Multiplication by 3
- Division by 2

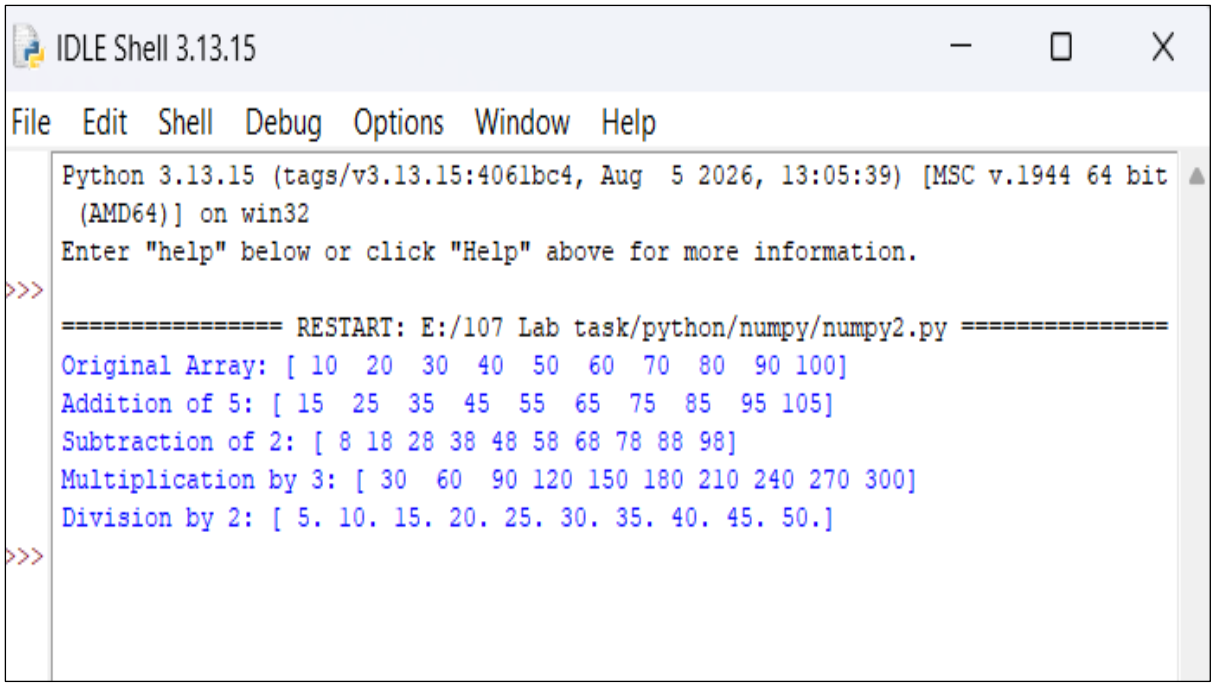
```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

print("Original Array:", arr)

print("Addition of 5:", arr + 5)

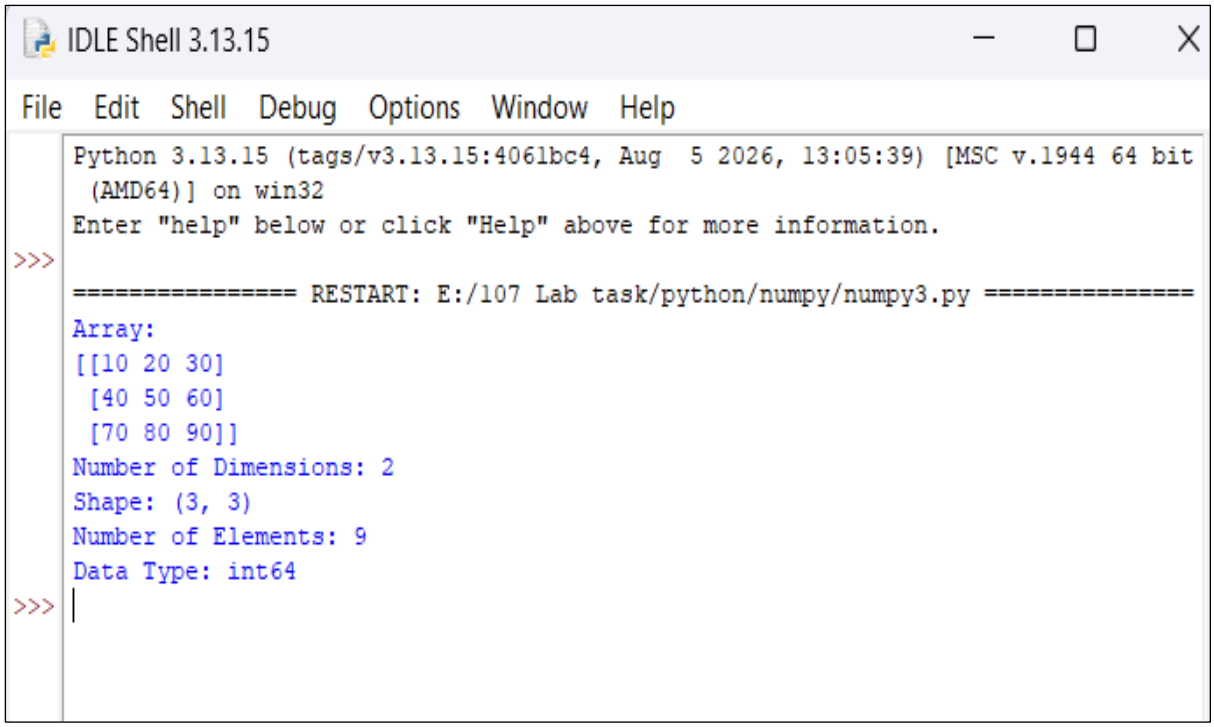
print("Subtraction of 2:", arr - 2)
print("Multiplication by 3:", arr * 3)
print("Division by 2:", arr / 2)
```



3. Create the following 2D array Perform:

- Display the array.
- Display its number of dimensions.
- Display its shape.
- Display the number of elements.
- Display the data type.

```
import numpy as np
arr = np.array([[10, 20, 30],
                [40, 50, 60],
                [70, 80, 90]])
print("Array:")
print(arr)
print("Number of Dimensions:", arr.ndim)
print("Shape:", arr.shape)
print("Number of Elements:", arr.size)
print("Data Type:", arr.dtype)
```



4. Create a NumPy array Perform:

- Display the first element.
- Display the last element.
- Display the third element.
- Display elements from index 1 to 4.
- Display the array in reverse order.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

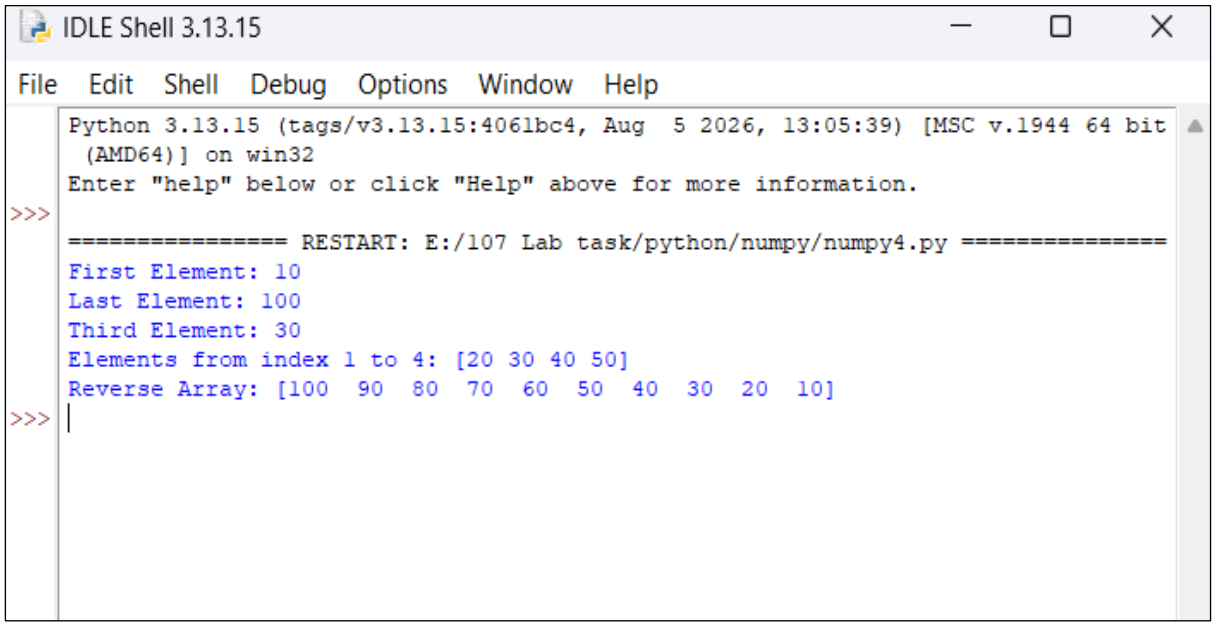

print("First Element:", arr[0])

print("Last Element:", arr[-1])


print("Third Element:", arr[2])
```

print("Elements from index 1 to 4:", arr[1:5])

print("Reverse Array:", arr[::-1])



5.Write a program to create:

- A 1D NumPy array containing five zeros.
- A 1D NumPy array containing five ones.
- A 3 × 3 matrix of zeros.
- A 3 × 3 matrix of ones.

import numpy as np

zeros_array = np.zeros(5, dtype=int)

print("1D Array of Five Zeros:")

print(zeros_array)

ones_array = np.ones(5, dtype=int)

print("\n1D Array of Five Ones:")

print(ones_array)

zeros_matrix = np.zeros((3, 3), dtype=int)

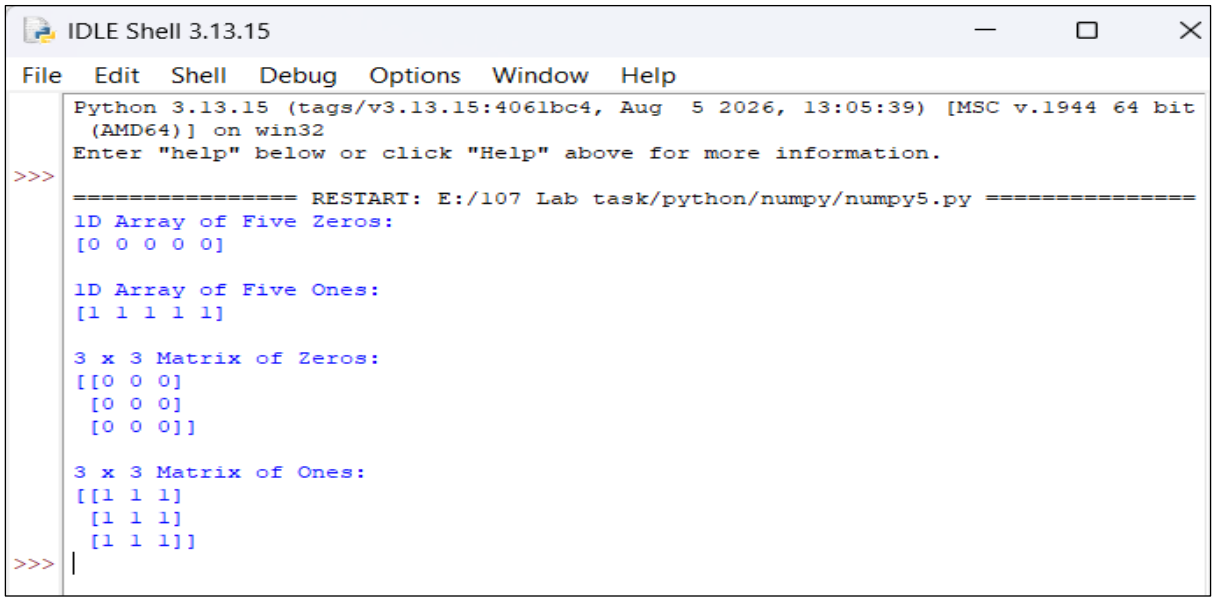
print("\n3 x 3 Matrix of Zeros:")

print(zeros_matrix)

ones_matrix = np.ones((3, 3), dtype=int)

print("\n3 x 3 Matrix of Ones:")

print(ones_matrix)



6. Create a 2D NumPy array containing marks of 5 students in 3 subjects:

Python Java DBMS

Perform:

- Calculate total marks of each student.
- Calculate average marks of each student.
- Find the highest mark in each subject.
- Find the lowest mark in each subject.
- Find the overall average marks.

```
import numpy as np

marks = np.array([

  [85, 90, 80],

  [75, 88, 92],

  [60, 70, 65],

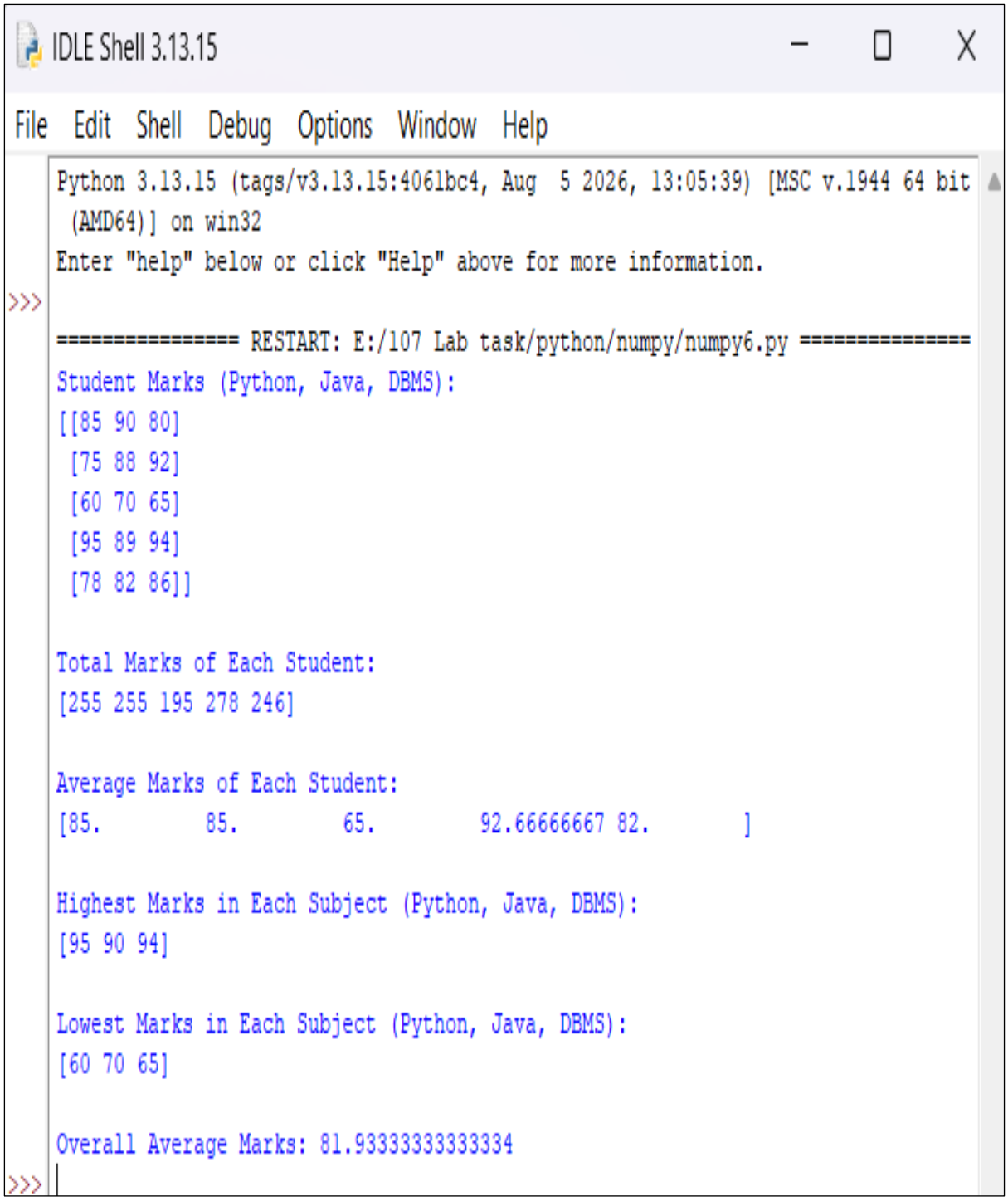
  [95, 89, 94],

  [78, 82, 86]

])
```

```
print("Student Marks (Python, Java, DBMS):")
print(marks)
total = np.sum(marks, axis=1)
print("\nTotal Marks of Each Student:")
print(total)
average = np.mean(marks, axis=1)
print("\nAverage Marks of Each Student:")
print(average)
highest = np.max(marks, axis=0)
print("\nHighest Marks in Each Subject (Python, Java, DBMS):")
print(highest)

lowest = np.min(marks, axis=0)
print("\nLowest Marks in Each Subject (Python, Java, DBMS):")
print(lowest)
overall_average = np.mean(marks)
print("\nOverall Average Marks:", overall_average)
```



7. Create a NumPy array containing salaries of 10 employees.

Perform the following:

1. Display all salaries.
2. Find the highest salary.
3. Find the lowest salary.
4. Calculate the average salary.

5. Display salaries greater than ₹40,000.
6. Display salaries less than ₹30,000.
7. Sort salaries in ascending order.
8. Sort salaries in descending order.
9. Count how many employees have salary greater than ₹50,000.

Add a 10% increment to all salaries and display the updated salaries

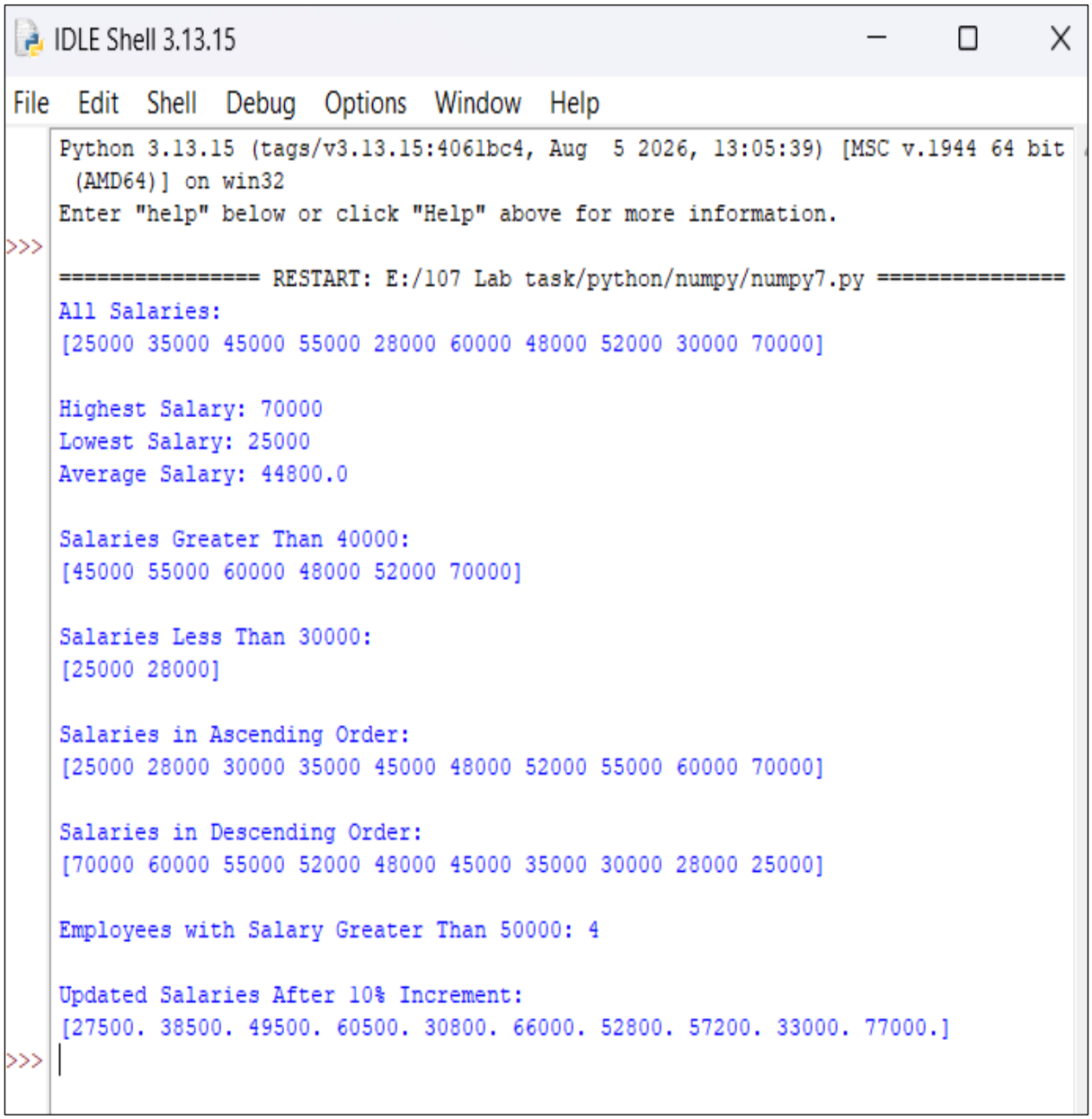
```
import numpy as np
salaries = np.array([25000, 35000, 45000, 55000, 28000,
                     60000, 48000, 52000, 30000, 70000])
print("All Salaries:")
print(salaries)
print("\nHighest Salary:", np.max(salaries))
rint("Lowest Salary:", np.min(salaries))
print("Average Salary:", np.mean(salaries))
print("\nSalaries Greater Than 40000:")
print(salaries[salaries > 40000])
print("\nSalaries Less Than 30000:")
print(salaries[salaries < 30000])

print("\nSalaries in Ascending Order:")
print(np.sort(salaries))
print("\nSalaries in Descending Order:")
print(np.sort(salaries)[::-1])

count = np.sum(salaries > 50000)
print("\nEmployees with Salary Greater Than 50000:", count)
updated_salaries = salaries * 1.10
```

```
print("\nUpdated Salaries After 10% Increment:")

print(updated_salaries)
```



8. Create a 4 × 5 NumPy array containing numbers from 1 to 20.

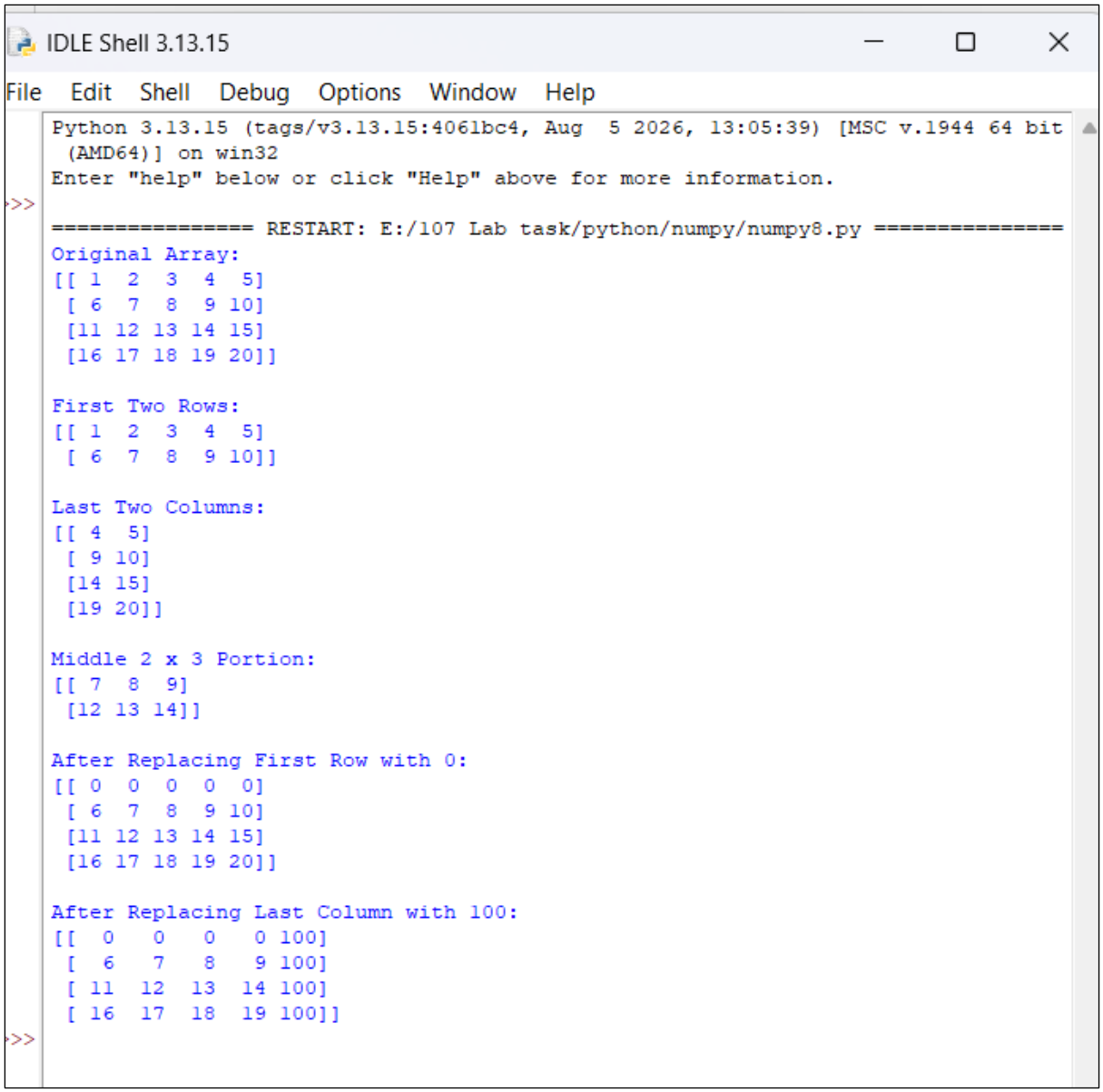
Perform:

- Extract the first two rows.
- Extract the last two columns.
- Extract the middle 2 × 3 portion.
- Replace all elements in the first row with 0.

Replace the last column with 100.

```
import numpy as np


arr = np.arange(1, 21).reshape(4, 5)
print("Original Array:")
print(arr)
print("\nFirst Two Rows:")
print(arr[:2, :])
print("\nLast Two Columns:")
print(arr[:, -2:])
print("\nMiddle 2 x 3 Portion:")
print(arr[1:3, 1:4])
arr[0, :] = 0
print("\nAfter Replacing First Row with 0:")
print(arr)
arr[:, -1] = 100
print("\nAfter Replacing Last Column with 100:")
print(arr)
```



9. Create a NumPy array containing marks of 20 students.

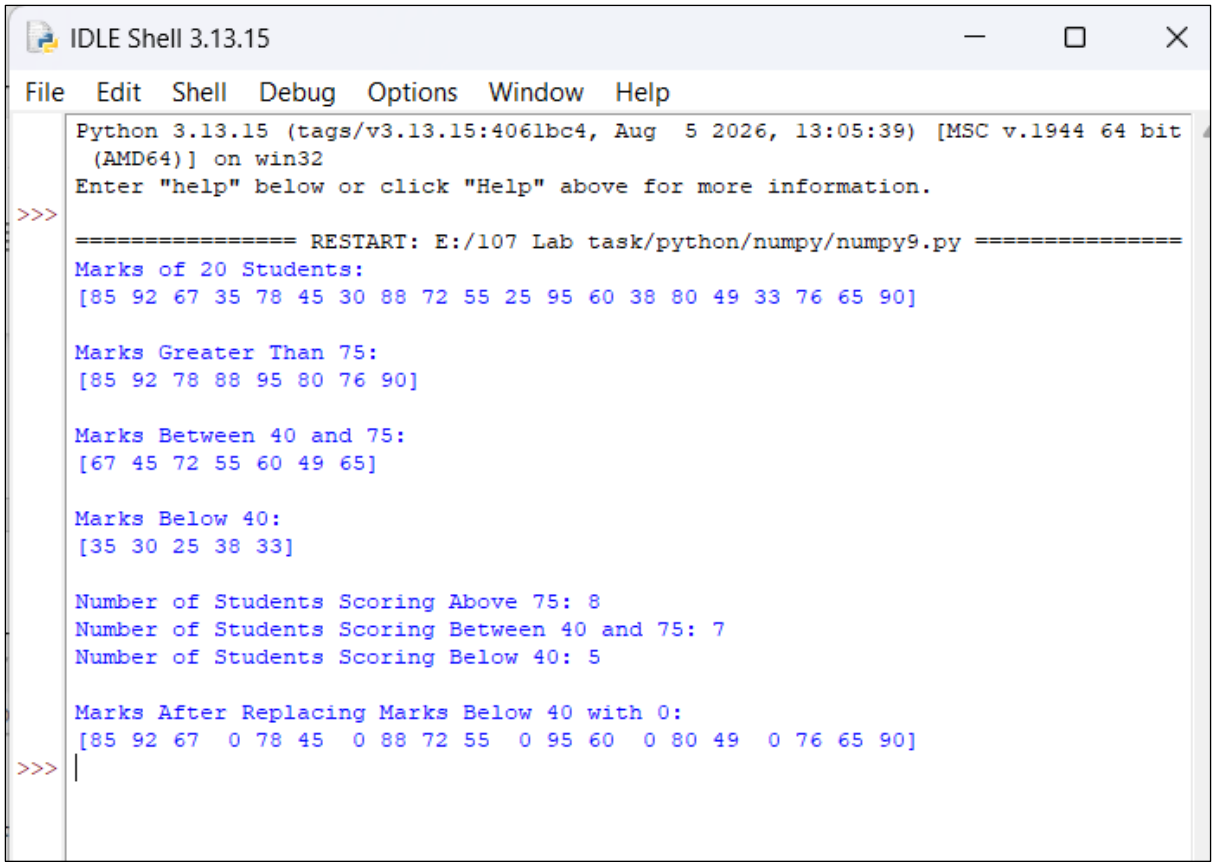
Perform:

- Display marks greater than 75.
- Display marks between 40 and 75.
- Display marks below 40.
- Count students in each category.
- Replace marks below 40 with 0.

```
import numpy as np
marks = np.array([85, 92, 67, 35, 78, 45, 30, 88, 72, 55,
                  25, 95, 60, 38, 80, 49, 33, 76, 65, 90])

print("Marks of 20 Students:")
print(marks)
above_75 = marks[marks > 75]
print("\nMarks Greater Than 75:")
print(above_75)

between_40_75 = marks[(marks >= 40) & (marks <= 75)]
print("\nMarks Between 40 and 75:")
print(between_40_75)
below_40 = marks[marks < 40]
print("\nMarks Below 40:")
print(below_40)
print("\nNumber of Students Scoring Above 75:", len(above_75))
print("Number of Students Scoring Between 40 and 75:", len(between_40_75))
print("Number of Students Scoring Below 40:", len(below_40))
marks[marks < 40] = 0
print("\nMarks After Replacing Marks Below 40 with 0:")
print(marks)
```



10. Create two 3 × 3 matrices.

Perform:

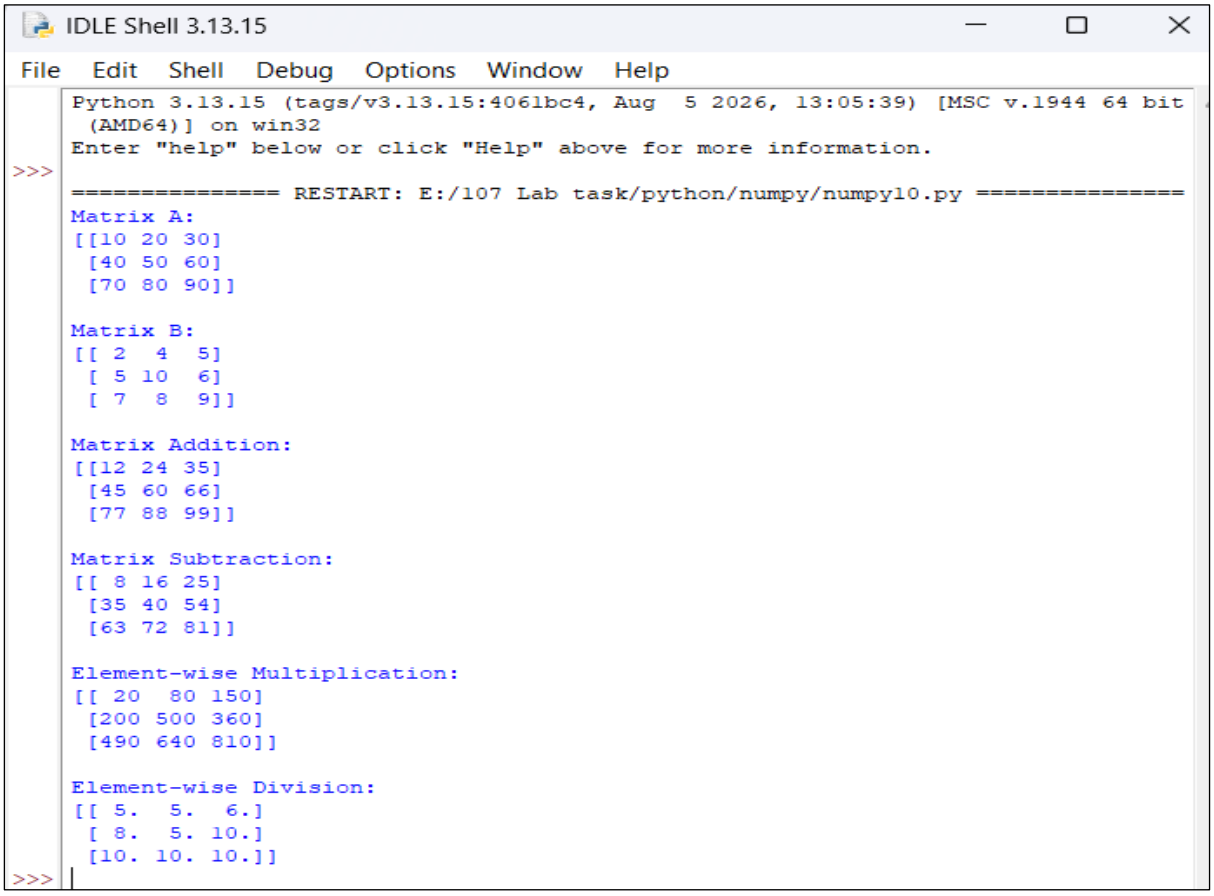
- Matrix addition.
- Matrix subtraction.
- Element-wise multiplication.
- Element-wise division.

Display the result of each operation

```
import numpy as np
A = np.array([[10, 20, 30],
              [40, 50, 60],
              [70, 80, 90]])
B = np.array([[2, 4, 5],
              [5, 10, 6],
```

```
        [7, 8, 9]])

print("Matrix A:")
print(A)
print("\nMatrix B:")
print(B)
print("\nMatrix Addition:")
print(A + B)
print("\nMatrix Subtraction:")
print(A - B)
print("\nElement-wise Multiplication:")
print(A * B)
print("\nElement-wise Division:")
print(A / B)
```



11. Create a NumPy array containing student marks where some values are represented by -1.

Example: marks = np.array([78, 65, -

1, 89, 56, -1, 72])

Perform:

- Find the positions of -1.
- Count the invalid values.
- Replace -1 with the average of valid marks.
- Display the updated array.

```
import numpy as np

marks = np.array([78, 65, -1, 89, 56, -1, 72])

print("Original Marks:")

print(marks)

positions = np.where(marks == -1)[0]

print("\nPositions of -1:")

print(positions)

count = np.sum(marks == -1)

print("\nNumber of Invalid Values:", count)

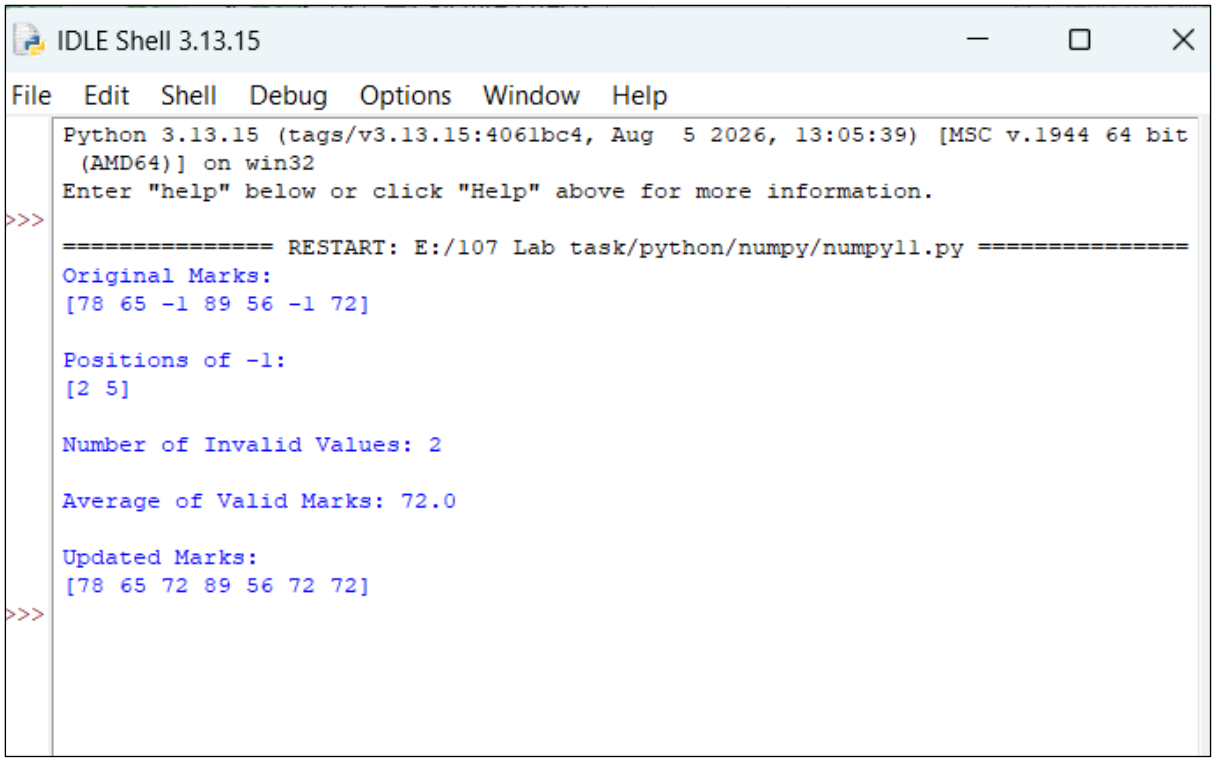
average = np.mean(marks[marks != -1])

print("\nAverage of Valid Marks:", average)

marks[marks == -1] = average

print("\nUpdated Marks:")

print(marks)
```



12.Create a 2D NumPy array representing sales of 5 products over 4 months.

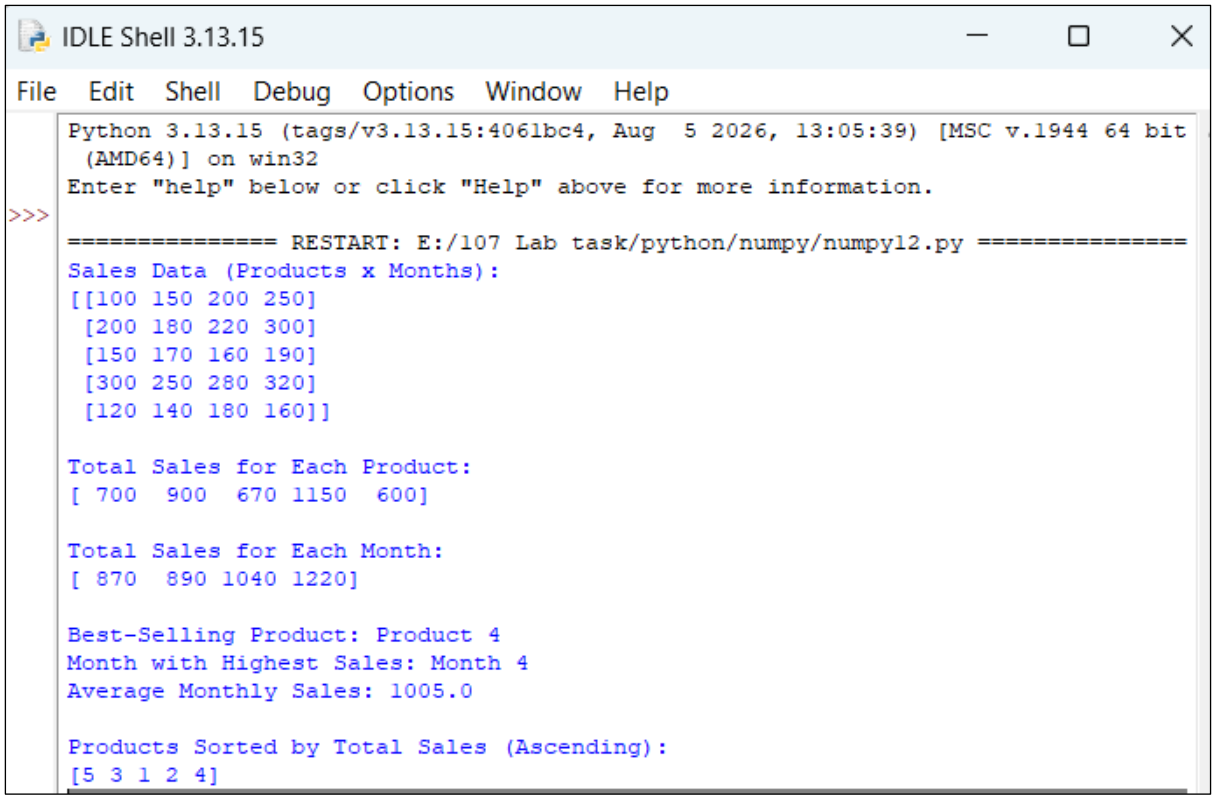
Perform:

- Calculate total sales for each product.
- Calculate total sales for each month.
- Find the best-selling product.
- Find the month with the highest sales.
- Calculate average monthly sales.
- Sort products according to total sales.

```
import numpy as np
sales = np.array([
  [100, 150, 200, 250],
  [200, 180, 220, 300],
  [150, 170, 160, 190],
  [300, 250, 280, 320],
```

```
    [120, 140, 180, 160]
])

print("Sales Data (Products x Months):")
print(sales)
product_totals = np.sum(sales, axis=1)
print("\nTotal Sales for Each Product:")
print(product_totals)
monthly_totals = np.sum(sales, axis=0)
print("\nTotal Sales for Each Month:")
print(monthly_totals)
best_product = np.argmax(product_totals) + 1
print("\nBest-Selling Product: Product", best_product)
best_month = np.argmax(monthly_totals) + 1
print("Month with Highest Sales: Month", best_month)
average_monthly_sales = np.mean(monthly_totals)
print("Average Monthly Sales:", average_monthly_sales)
sorted_products = np.argsort(product_totals) + 1
print("\nProducts Sorted by Total Sales (Ascending):")
print(sorted_products)
```



13.Create a 5 × 5 NumPy matrix using random integers between 1 and 100.

Perform:

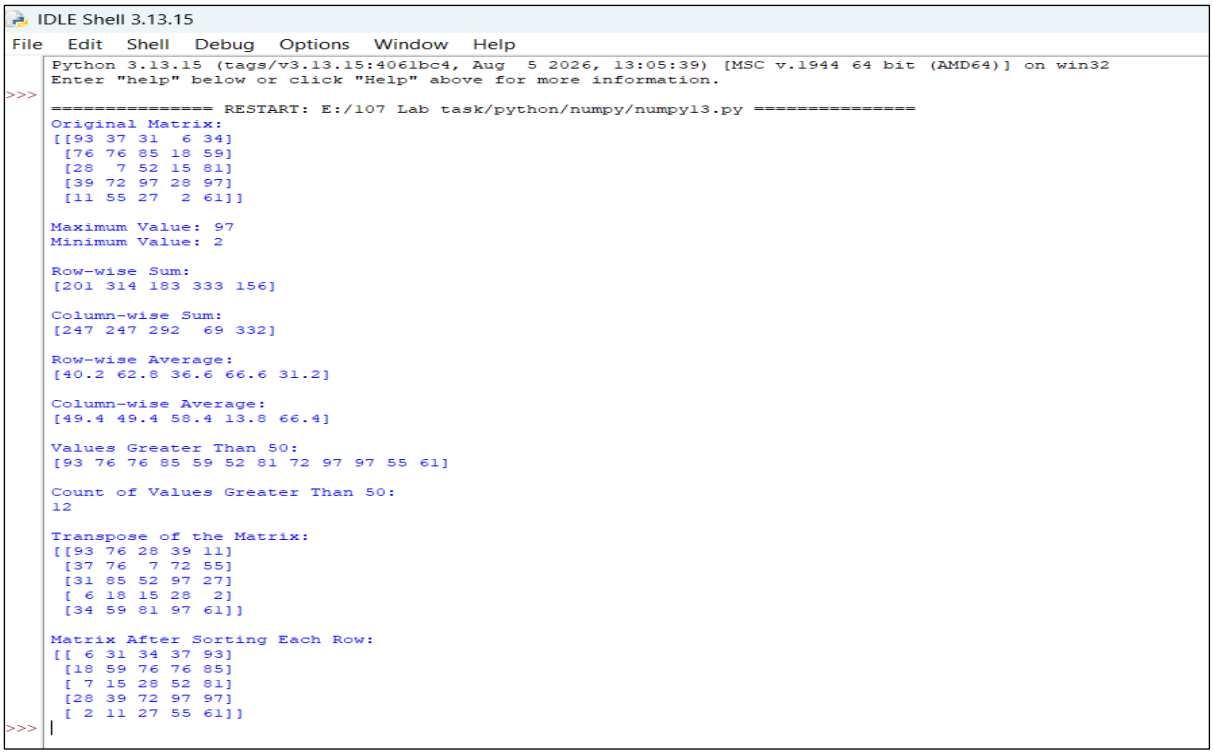
1. Display the matrix.
2. Find the maximum and minimum values.
3. Calculate row-wise sum.
4. Calculate column-wise sum. 5. Calculate row-wise average.
6. Calculate column-wise average.
7. Find all values greater than 50.
8. Count values greater than 50.
9. Find the transpose of the matrix.
10. Sort the values of each row.

```
import numpy as np

matrix = np.random.randint(1, 101, size=(5, 5))
```

```
print("Original Matrix:")
print(matrix)

print("\nMaximum Value:", np.max(matrix))
print("Minimum Value:", np.min(matrix))
print("\nRow-wise Sum:")
print(np.sum(matrix, axis=1))
print("\nColumn-wise Sum:")
print(np.sum(matrix, axis=0))
print("\nRow-wise Average:")
print(np.mean(matrix, axis=1))
print("\nColumn-wise Average:")
print(np.mean(matrix, axis=0))
print("\nValues Greater Than 50:")
print(matrix[matrix > 50])
print("\nCount of Values Greater Than 50:")
print(np.sum(matrix > 50))
print("\nTranspose of the Matrix:")
print(matrix.T)
print("\nMatrix After Sorting Each Row:")
print(np.sort(matrix, axis=1))
```


IDLE Shell 3.13.15
File Edit Shell Debug Options Window Help
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug 5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: E:/107 Lab task/python/numpy/numpy13.py ==========
Original Matrix:
[[93 37 31 6 34]
[76 76 85 18 59]
[28 7 52 15 81]
[39 72 97 28 97]
[11 55 27 2 61]]
Maximum Value: 97
Minimum Value: 2
Row-wise Sum:
[201 314 183 333 156]
Column-wise Sum:
[247 247 292 69 332]
Row-wise Average:
[40.2 62.8 36.6 66.6 31.2]
Column-wise Average:
[49.4 49.4 58.4 13.8 66.4]
Values Greater Than 50:
[93 76 76 85 59 52 81 72 97 97 55 61]
Count of Values Greater Than 50:
12
Transpose of the Matrix:
[[93 76 28 39 11]
[37 76 7 72 55]
[31 85 52 97 27]
[6 18 15 28 2]
[34 59 81 97 61]]
Matrix After Sorting Each Row:
[[6 31 34 37 93]
[18 59 76 76 85]
[7 15 28 52 81]
[28 39 72 97 97]
[2 11 27 55 61]]
>>>